

Comparative Analysis of Shortest Path Algorithms: Floyd-Warshall, Johnson's, and Dijkstra

Introduction

In network routing and graph theory, finding the shortest path between nodes is a fundamental problem. While Dijkstra's and Bellman-Ford algorithms solve the single-source shortest path problem, there are scenarios where we need to find shortest paths between all pairs of vertices. This report examines two prominent all-pairs shortest path algorithms—Floyd-Warshall and Johnson's algorithm—comparing them with repeated applications of Dijkstra's algorithm, and analyzing their efficiency across different network topologies.

Floyd-Warshall Algorithm

Algorithm Overview

The Floyd-Warshall algorithm is a dynamic programming approach that computes shortest paths between all pairs of vertices in a weighted directed graph. Unlike single-source algorithms, it simultaneously finds the shortest path distance between every possible pair of nodes in the network.

Working Mechanism

The algorithm operates on the fundamental principle of considering intermediate vertices. For any pair of vertices (i, j) , it examines whether routing through an intermediate vertex k produces a shorter path than the direct path currently known.

The algorithm maintains a distance matrix where $\text{dist}[i][j]$ represents the shortest known distance from vertex i to vertex j . It iteratively improves this matrix by considering each vertex as a potential intermediate point. The core operation can be expressed as:

$$\text{dist}[i][j] = \text{minimum of } (\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$$

The algorithm proceeds through three nested loops: the outermost loop iterates through each potential intermediate vertex k , while the two inner loops examine all pairs of vertices (i, j) . After considering all n vertices as intermediates, the distance matrix contains all shortest path distances.

Complexity Analysis

The Floyd-Warshall algorithm has a time complexity of $O(V^3)$, where V represents the number of vertices. This cubic time complexity arises from the three nested loops, each iterating V times. The space complexity is $O(V^2)$ for storing the distance matrix.

Advantages and Limitations

The algorithm can handle graphs with negative edge weights, making it more versatile than Dijkstra's algorithm. However, it cannot handle negative cycles, as these would make the concept of "shortest path" undefined. The algorithm is remarkably simple to implement and naturally produces a complete distance matrix as output.

Johnson's Algorithm

Algorithm Overview

Johnson's algorithm represents a sophisticated hybrid approach that combines Bellman-Ford and Dijkstra's algorithms to solve the all-pairs shortest path problem efficiently, particularly for sparse graphs.

Working Mechanism

The algorithm operates in several distinct phases. First, it introduces a new auxiliary vertex that connects to all existing vertices with zero-weight edges. From this new vertex, it runs the Bellman-Ford algorithm to compute a reweighting function. This step also serves to detect any negative cycles in the graph.

The reweighting function transforms all edge weights to non-negative values while preserving the shortest paths. This transformation is crucial because it allows the use of Dijkstra's algorithm, which is more efficient than Bellman-Ford but cannot handle negative weights.

After reweighting, the algorithm runs Dijkstra's algorithm from each vertex in the modified graph. Finally, it adjusts the computed distances back to reflect the original edge weights, producing the final all-pairs shortest path matrix.

Complexity Analysis

Johnson's algorithm has a time complexity of $O(V^2 \log V + VE)$ when using a binary heap for Dijkstra's implementation, or $O(V^2 + VE)$ with a Fibonacci heap. The space complexity is $O(V^2)$ for storing the result matrix. The Bellman-Ford phase contributes $O(VE)$ time, while running Dijkstra from each vertex contributes $O(V^2 \log V)$ for sparse graphs.

Advantages and Limitations

This algorithm excels with sparse graphs and can handle negative edge weights. It combines the negative-weight handling capability of Bellman-Ford with the efficiency of Dijkstra's algorithm for the bulk of the computation.

Key Differences from Dijkstra's Algorithm

Fundamental Problem Scope

Dijkstra's algorithm solves the single-source shortest path problem, finding shortest paths from one source vertex to all other vertices. In contrast, both Floyd-Warshall and Johnson's algorithms solve the all-pairs problem, finding shortest paths between every pair of vertices.

Handling Negative Weights

Dijkstra's algorithm cannot correctly handle negative edge weights because its greedy strategy assumes that once a vertex is finalized, no shorter path to it will be found. Floyd-Warshall and Johnson's algorithms can both handle negative weights, though neither can handle negative cycles.

Algorithmic Approach

Dijkstra employs a greedy strategy, always selecting the nearest unvisited vertex. Floyd-Warshall uses dynamic programming, systematically considering all possible intermediate vertices. Johnson's uses a hybrid approach, first reweighting the graph and then applying Dijkstra's greedy strategy multiple times.

Complexity Considerations

A single run of Dijkstra's algorithm has complexity $O((V+E) \log V)$ with a binary heap. Running it from all vertices (repeated Dijkstra) yields $O(V(V+E) \log V)$, which simplifies to $O(V^2 \log V + VE \log V)$. This complexity must be compared against Floyd-Warshall's $O(V^3)$ and Johnson's $O(V^2 \log V + VE)$.

Efficiency Analysis: Topology and Density Considerations

Graph Density Definition

A graph's density significantly impacts algorithm performance. We define a graph as sparse when the number of edges E is much smaller than V^2 , typically $E = O(V)$. A dense graph has E approaching V^2 , meaning nearly every pair of vertices is connected.

Floyd-Warshall Efficiency Conditions

Floyd-Warshall becomes more efficient than repeated Dijkstra primarily in dense graphs. For dense graphs where $E \approx V^2$, repeated Dijkstra's complexity becomes $O(V^3 \log V)$, while Floyd-Warshall remains $O(V^3)$. Floyd-Warshall wins by eliminating the logarithmic factor.

The algorithm is particularly suitable for small to medium-sized networks, typically where $V < 500$ vertices. Beyond this size, even with dense graphs, the cubic complexity becomes prohibitively expensive. Additionally, when implementation simplicity is valued and the network contains negative edge weights, Floyd-Warshall is an excellent choice.

Johnson's Algorithm Efficiency Conditions

Johnson's algorithm excels with sparse graphs where E is significantly smaller than V^2 . For such graphs, the VE term in Johnson's complexity remains small, while repeated Dijkstra's VE

$\log V$ term grows larger. Johnson's complexity of $O(V^2 \log V + VE)$ becomes superior to repeated Dijkstra's $O(V^2 \log V + VE \log V)$.

This algorithm is particularly effective for large networks with relatively few edges. When the graph has thousands of vertices but each vertex connects to only a handful of others, Johnson's algorithm significantly outperforms both alternatives. It is also the algorithm of choice when negative edge weights are present in a sparse graph.

Repeated Dijkstra Efficiency Conditions

Repeated Dijkstra remains competitive for moderately sparse graphs without negative edges. When E is approximately $O(V \log V)$, its performance is comparable to Johnson's but with simpler implementation. Additionally, if not all pairs of shortest paths are needed, repeated Dijkstra offers flexibility to compute only required paths.

Quantitative Comparison Examples

Small Dense Network

Consider a network with 100 vertices and 5,000 edges (a moderately dense graph). Repeated Dijkstra performs approximately $100 \times 5,000 \times \log(100) \approx 3.3$ million operations. Floyd-Warshall performs $100^3 = 1$ million operations. In this scenario, Floyd-Warshall is clearly more efficient.

Large Sparse Network

For a network with 1,000 vertices and 5,000 edges (a sparse graph), repeated Dijkstra performs approximately $1,000 \times 5,000 \times \log(1,000) \approx 50$ million operations. Johnson's algorithm performs approximately $1,000,000 \times \log(1,000) + 5,000,000 \approx 15$ million operations. Floyd-Warshall would perform 1 billion operations, making it completely impractical. Johnson's algorithm provides a threefold improvement over repeated Dijkstra.

Very Large Dense Network

For a network with 1,000 vertices and 400,000 edges (dense), repeated Dijkstra performs approximately 4 billion operations, while Floyd-Warshall performs 1 billion operations. However, both become computationally expensive at this scale, and practical implementations might require distributed computing or approximation algorithms.

Practical Selection Guidelines

Choose Floyd-Warshall When:

- Network size is moderate ($V \leq 500$ approximately)
- Graph is dense with $E \geq V^2/\log V$
- Negative edge weights are present

- Simple, straightforward implementation is prioritized
- Complete distance matrix is required as output
- Memory for V^2 matrix is available

Choose Johnson's Algorithm When:

- Graph is sparse with E much less than V^2
- Network is large ($V > 500$)
- Negative edge weights need to be handled
- Computational efficiency is critical
- More complex implementation is acceptable

Choose Repeated Dijkstra When:

- Graph is sparse or moderately sparse
- No negative edge weights exist
- Only specific pairs of shortest paths are needed
- Incremental computation is valuable
- Graph is dynamic and paths are computed on-demand

Conclusion

The choice between Floyd-Warshall, Johnson's, and repeated Dijkstra algorithms depends critically on network characteristics. Graph density emerges as the primary factor: Floyd-Warshall dominates for dense graphs below moderate size, Johnson's excels for large sparse graphs, and repeated Dijkstra remains competitive for moderately sparse graphs without negative weights.

Understanding these algorithmic tradeoffs enables network engineers and computer scientists to select the most appropriate solution for their specific topology, ensuring optimal performance in routing protocols, network analysis, and graph-based applications. The theoretical complexity analysis provides clear guidance, but practical considerations such as implementation complexity, memory constraints, and specific use-case requirements should also inform the final decision.